Formalisme exact du modèle effectivement implémenté `eLLM_v0`

Spécification mathématique et contrat logiciel traçable

Version 0.2.0 – 21 octobre 2026

Résumé

Ce document formalise l'artefact actuel, plutôt qu'une architecture souhaitée mais non codée. Il sépare les éléments du socle des empreintes, les choix d'implémentation, les propriétés démontrées par construction, les propriétés vérifiées sur un domaine fini et les hypothèses empiriques. Le prédicteur est une chaîne de Markov à ordre variable réalisée par des n-grammes interpolés avec lissage ; la couche observationnelle construit des signatures exactes à partir de sondes suffixes emboîtées. L'ensemble est complété par la factorisation sur fibres finies, la mémoire optionnelle, la recherche, l'imagination, les diagnostics et le pipeline HTTP/Web. Les fonctions et plages de lignes sont référencées dans `prouves_eLLM_v0.json`.

Table des matières

1 Description **2**
1.1 Objet formel livré . . . 2
1.2 Statuts épistémiques . . . 2

2 Architecture **2**

3 Pipeline complet **2**
3.1 Objet et tokenisation . . . 2
3.2 Comptages conditionnels . . . 3
3.3 Apprentissage et fonction de perte . . . 3
3.4 Sondes et empreintes . . . 3
3.5 Indiscernabilité, fibres, quotients et raffinement . . . 3
3.6 Seuil, donnée de bord et pseudoultramétrique . . . 4
3.7 Mesure empirique des seuils . . . 4
3.8 Fermeture observationnelle . . . 4
3.9 Capacité effective et dimension effective . . . 4
3.10 Factorisation et transport . . . 4
3.11 Maillage relationnel . . . 5
3.12 Mémoire, dialogue et génération . . . 5
3.13 Recherche et imagination . . . 5
3.14 Diagnostics . . . 5
3.15 Interface/API . . . 5

4 Commandes d'exécution **5**

5 Résultats **6**

6 Interprétations des résultats **6**

7 Conséquences théoriques, conceptuelles, opérationnelles **6**
7.1 Théoriques 6
7.2 Conceptuelles 6
7.3 Opérationnelles 7

8 Conclusions **7**

9 Statut du modèle **7**

1 Description

1.1 Objet formel livré

On note $\mathcal{V}$ le vocabulaire appris, $\mathcal{V}^*$ les séquences de tokens, D_{tr} le train, D_{va} la validation et D_{te} le test. La distribution de l'artefact n'est pas produite par un Transformer ; elle vient de compteurs conditionnels et d'une interpolation récursive. La couche d'empreintes est un observateur distinct appliqué à la séquence tokenisée et à la distribution prédictive.

1.2 Statuts épistémiques

BASE Notions du socle : objet, contexte, sondes, fonctionnel, empreinte, échelle, indiscernabilité, fibre, quotient, seuil, fermeture et factorisation.

DÉRIVÉ Propriétés mathématiques sous hypothèses explicites : relation d'équivalence de l'égalité de signatures, raffinement par inclusion de sondes, constance sur fibres iff factorisation, loi ultramétrique d'un seuil à minimum double.

INGÉNIERIE Choix explicitement codés : n-grammes interpolés, suffixes comme sondes, sérialisation JSON, session/mémoire locale, endpoints et contrôles de requête.

EMPIRIQUE Affirmations qui demandent des expériences : compétence linguistique, fidélité sémantique, généralisation comportementale, utilité réelle des trajectoires.

2 Architecture

Le tuple opérationnel peut être noté

$$\mathfrak{E}_\theta = (\mathcal{V}, \mathrm{Tok}, C_0, \ldots, C_m, \alpha, \mathsf{Pred}, \mathsf{Obs}, \mathsf{Quot}, \Theta, \mathsf{Mem}, \mathsf{Search}, \mathsf{Imagine}, \mathsf{API}, \mathsf{UI}).$$

Tous ces symboles décrivent le code présent. Ils ne signifient pas que la sémantique complète est représentée ou que la théorie est démontrée universellement.

3 Pipeline complet

3.1 Objet et tokenisation

Une séquence textuelle $s \in \Sigma^*$ est convertie par un tokenizer déterministe $\tau(s) = (w_1, \ldots, w_n) \in \mathcal{V}^*$. Le vocabulaire est construit depuis D_{tr} uniquement ; les unités spéciales incluent BOS/EOS/UNK. Tous les observateurs internes utilisent le même flux de tokens canonique que le prédicteur, afin que l'équivalence d'empreinte ne porte pas par erreur sur une tokenisation différente.

3.2 Comptages conditionnels

Pour l'ordre $k \in \{0, \ldots, m\}$, soit c_k un contexte suffixe de longueur k, $N_k(c_k)$ le nombre de cibles entraînées vues après ce contexte, et $C_k(c_k, w)$ le nombre d'occurrences du token cible w. Le niveau 0 est

$$P_0(w) = \frac{C_0(\varnothing, w) + \alpha}{N_0 + \alpha|\mathcal{V}|}, \qquad \alpha > 0.$$

Pour $k \geq 1$, avec c_{k-1} obtenu en retirant le token le plus ancien de c_k,

$$P_k(w \mid c_k) = \frac{C_k(c_k, w) + \alpha P_{k-1}(w \mid c_{k-1})}{N_k(c_k) + \alpha}.$$

Si le contexte n'a jamais été vu, le compte est nul et la distribution retombe sur l'ordre inférieur. La forme d'interpolation assure la normalisation par récurrence dès que chaque P_{k-1} est normalisée et que le dénominateur est cohérent avec la somme des comptes. Le lissage α doit être strictement positif et fini.

3.3 Apprentissage et fonction de perte

Le corpus est séparé par document en partitions disjointes D_{tr}, D_{va}, D_{te}. Pour les tokens cibles y_i et contextes x_i, la perte moyenne est

$$\mathcal{L}(D; \theta) = -\frac{1}{N_D} \sum_{i=1}^{N_D} \log P_\theta(y_i \mid x_i).$$

La perplexité est $\mathrm{PPL}(D) = \exp(\mathcal{L}(D; \theta))$, et la top-1 est la fraction dont la cible égale $\arg\max_w P_\theta(w \mid x_i)$. L'implémentation n'utilise pas de gradient neuronal : « entraînement » signifie reconstruction des tables de fréquences depuis D_{tr}.

3.4 Sondes et empreintes

Le domaine opérationnel observé est $\mathcal{X} = \mathcal{V}^*$ ou, dans l'analyse finie, un sous-ensemble explicite $\mathcal{U} \subset \mathcal{X}$. La famille de sondes à résolution k est

$$\mathcal{F}_k = \{f_1, \ldots, f_k\}, \qquad f_j(x) = \text{suffixe de longueur } j \text{ de } x,$$

avec convention de suffixe définie dans `ProbeFamily.value`. Comme $\mathcal{F}_k \subseteq \mathcal{F}_{k+1}$, les signatures sont emboîtées :

$$I_k(x) = (f_1(x), \ldots, f_k(x)), \quad I_0(x) = ().$$

La signature discrète est une empreinte exacte pour cette famille précise. Une empreinte continue distincte contient des statistiques de prédiction (notamment distribution, entropie et tokens supérieurs) ; elle n'est pas utilisée pour décider l'égalité exacte des classes.

3.5 Indiscernabilité, fibres, quotients et raffinement

$$x \sim_k y \iff I_k(x) = I_k(y), \qquad [x]_k = \{y \in \mathcal{U} : I_k(y) = I_k(x)\}, \qquad Q_k = \mathcal{U}/\sim_k .$$

L'égalité est réflexive, symétrique et transitive ; donc $\sim_k$ est une relation d'équivalence. L'inclusion $\mathcal{F}_k \subseteq \mathcal{F}_{k+1}$ implique $x \sim_{k+1} y \Rightarrow x \sim_k y$, d'où le raffinement des partitions. Cette propriété est exacte par construction, mais son intérêt sémantique dépend du choix des sondes.

3.6 Seuil, donnée de bord et pseudoultramétrique

Pour une paire de séquences, le code prend le niveau maximal dont les signatures restent égales :

$$\rho_m(x, y) = \max\{k \in \{0, \dots, m\} : I_k(x) = I_k(y)\}.$$

Dans cette implémentation, deux séquences exactement identiques restent égales jusqu'à m, y compris si elles sont plus courtes que m ; ce cas limite a été couvert par un test de régression. La valeur du seuil ne suffit pas à tous les régimes continus possibles ; pour un niveau discret, `boundary_status` indique si l'égalité existe au niveau critique. Comme le seuil satisfait la loi du minimum double sur les signatures emboîtées,

$$\rho(x, z) \geq \min\{\rho(x, y), \rho(y, z)\},$$

la transformation $d(x, y) = \exp(-\rho(x, y))$ est ultramétrique au sens de l'inégalité faible, mais peut rester une pseudoultramétrique sur le domaine original parce que deux objets différents peuvent avoir la même signature maximale.

3.7 Mesure empirique des seuils

Pour $U = |\mathcal{U}| \geq 2$, l'histogramme empirique des seuils est

$$\nu_U = \binom{U}{2}^{-1} \sum_{\{x,y\} \subset \mathcal{U}} \delta_{\rho(x,y)}.$$

La résolution cumulée est calculée suivant la convention stricte enregistrée par le code et son résultat ; toute interprétation de ses bornes dépend de cette convention et de l'ensemble fini échantillonné.

3.8 Fermeture observationnelle

Pour un ensemble de sondes G, la fermeture construite sur les sondes disponibles contient une sonde f si et seulement si sa réponse observée est constante sur les fibres de I_G : il existe une fonction ψ_f telle que

$$f(x) = \psi_f(I_G(x)) \quad \forall x \in \mathcal{U}.$$

Le test sur domaine fini donne une équivalence calculable par groupement des objets selon leur signature et recherche de conflits. L'univers effectivement fourni doit être documenté ; la fermeture calculée sur un échantillon ne démontre pas la fermeture sur $\mathcal{X}$ tout entier.

3.9 Capacité effective et dimension effective

La capacité effective compte les classes observationnellement distinctes parmi les sondes disponibles selon la définition opérationnelle de `effective_capacity` ; la dimension effective est la dimension affine des signatures représentées dans l'implémentation sur le domaine fini. Ces valeurs sont conditionnelles à la famille de probes et au domaine fourni. Elles ne signifient pas le nombre de concepts cognitifs internes.

3.10 Factorisation et transport

Pour une empreinte $I_G : \mathcal{U} \to L_G$ et une opération $T : \mathcal{U} \to Y$, la factorisation existe si et seulement si

$$I_G(x) = I_G(y) \Rightarrow T(x) = T(y).$$

Alors $\widetilde{T}(I_G(x)) = T(x)$ est bien définie sur $I_G(\mathcal{U})$. `transport.py` vérifie aussi la commutation d'une restriction entre deux signatures emboîtées, sous réserve que les opérations comparées soient définies sur les mêmes valeurs. Cela ne garantit pas que toute transition imaginée ou tout opérateur sémantique réel soit compatible.

3.11 Maillage relationnel

Un graphe de relations est construit sur les tokens/composantes observées et des arêtes pondérées par co-présence ou proximité opérationnelle. L'overlap/Jaccard mesure le recouvrement selon ce protocole ; il ne prouve pas une identité de sens. L'alignement de composants partagés dépend de l'identité canonique choisie dans le code.

3.12 Mémoire, dialogue et génération

La mémoire est un fichier JSON local optionnel. L'écriture persiste des items textuels et des représentations de mots-clés ; la lecture classe les candidats par chevauchement heuristique. Le prompt de chat réunit requête et derniers tours conservés par l'orchestrateur. La génération échantillonne la distribution conditionnelle, avec température et top-k ; EOS peut l'arrêter tôt. Les données de session et de mémoire n'étendent pas automatiquement les tables entraînées du prédicteur.

3.13 Recherche et imagination

La recherche détermine par beam search une liste de continuations classées d'après le score de log-probabilité normalisé. L'imagination génère plusieurs trajectoires avec graines distinctes et enregistre les tokens/réponses observationnelles par étape. Il s'agit d'exploration interne des continuations probabilistes, pas d'un simulateur de monde ni d'une preuve de raisonnement causal.

3.14 Diagnostics

Le vecteur de diagnostics sépare D_{obs}, D_{cap}, D_{res}, D_{search}, $D_{transfer}$ et $D_{constraint}$. Chaque composante dans le code est explicitement une mesure ou un proxy déclaré ; aucune ne représente à elle seule un déficit latent vrai, et certaines sont seulement heuristiques.

3.15 Interface/API

Le serveur HTTP standard library charge le checkpoint livré et orchestre le même `ChatEngine` que la CLI. Les routes permettent dialogue, recherche, imagination, diagnostic, transport, métriques et contrôle de mémoire. L'interface HTML/CSS/JavaScript locale ne dépend pas d'un service distant. Les gardes HTTP (prompt de 4000 caractères, génération de requête 1–2048, restrictions sur beam/count/body) sont des protections opérationnelles de service, séparées du modèle mathématique ; elles ne sont pas décrites comme capacités intrinsèques du prédicteur.

4 Commandes d'exécution

```
cd Dmodel_eLLM_v0
python run_eLLM_v0.py serve --host 127.0.0.1 --port 8765
python scripts/tune_and_train.py \
  --corpus data/corpus_fr.txt \
  --checkpoint checkpoints/model.json \
  --metrics checkpoints/training_metrics.json \
  --seed 2025 --contexts 1,2,3,4,5 \
  --alphas 0.35,1,3,10,30 --max-vocab 5000 \
  --selection-policy max_context_then_validation_nll
python -m unittest discover -s tests -v
node --check web/app.js
python run_eLLM_v0.py footprint "un exemple"
python run_eLLM_v0.py search \
  "Utilisateur : exemple Assistant :"
```

```
python run_eLLM_v0.py imagine \
  "Utilisateur : exemple Assistant :" --count 3 --seed 17
python run_eLLM_v0.py diagnose "Utilisateur : exemple Assistant :"
python run_eLLM_v0.py transport-check
```

5 Résultats

Le checkpoint est d'ordre 5, $\alpha = 10$, vocabulaire de 869 tokens, 64 documents d'apprentissage et 2701 cibles. Sur 8 documents/328 tokens du test gelé : NLL 5.38008, perplexité 217.0393, top-1 21.04% ; baseline unigramme même test : perplexité 449.9162, top-1 5.79%. Trois partitions seeds 2025/7/42 donnent les PPL test 217.039/295.967/290.436 et les baseline PPL 449.916/430.436/438.909. Le corpus source reste commun aux graines. L'audit structurel final, exécuté sur le manifeste canonique actuel de 80 états préfixes, donne des tailles de partitions 1/53/79/80/80/80 aux niveaux 0–5 ; zéro violation de raffinement, zéro incohérence seuil–filtration, 512000 triplets et zéro violation du minimum double. Le journal de régression P05 contient un contrôle antérieur sur un autre manifeste, dont les tailles étaient 1/56/80/80/80/80 ; les deux mesures sont conservées comme expériences distinctes, et seule la première est référencée comme canonique dans ce rapport. Le facteur prédictif est constant sur les fibres à l'ordre 5 sur le domaine de ce test et ne l'est pas à résolution 0. Le checkpoint reconstruit depuis la commande enregistrée a un hash identique au fichier distribué. Le test unitaire/HTTP passe 38 tests ; l'intégration HTTP/CLI, la syntaxe JS et le DOM Chromium ont réussi dans le protocole documenté. Le ZIP runtime testé après extraction propre porte le SHA-256 `77aca7017cf796dd095157946f9526623243bfe87afb9a73245ced86469975c6`. La suite exécutée depuis cette extraction passe 38/38 tests, l'aide CLI termine avec le code 0 et les huit contrôles du serveur réel réussissent (disponibilité, HTML, JS, chat, recherche, imagination, reset et effacement mémoire). Le résultat exact et son hash source sont consignés dans `Amodel_eLLM_v0/experiments/clean_extraction_portability.json` ; aucun service distant ni dépendance tierce n'est requis à l'exécution. Deux reconstructions runtime identiques sont documentées dans `Amodel_eLLM_v0/experiments/package_reproducibility.json`.

6 Interprétations des résultats

Le modèle s'exécute fidèlement à ses formules de comptage/interpolation et la couche d'empreinte vérifie sur domaine fini les propriétés construites. L'avantage de perplexité sur le baseline unigramme est réel sur les trois splits mesurés, mais les échantillons test sont petits et proviennent du même corpus synthétique/pédagogique. À contexte court, les prompts peuvent produire la même distribution ; le checkpoint livré vise un compromis conditionnel plutôt qu'une minimisation stricte de la seule NLL de validation. Ces mesures ne prouvent aucune compréhension, factualité ou supériorité générale.

7 Conséquences théoriques, conceptuelles, opérationnelles

7.1 Théoriques

La relation d'équivalence est exacte pour l'égalité des signatures ; l'inclusion des familles implique le raffinement ; la fermeture et la factorisation sont calculées sur des univers finis ; le seuil des signatures emboîtées a une cohérence hiérarchique. Les preuves concernent ces définitions et ces domaines.

7.2 Conceptuelles

Les coordonnées et signatures sont des observables, non des concepts par essence. Le graphe et la mémoire offrent une formalisation pratique du recouvrement, mais leur recouvrement n'est

pas une preuve de sémantique partagée. La généralisation comportementale doit être mesurée séparément de la généralisation observationnelle au sens de la distribution des seuils.

7.3 Opérationnelles

L'artefact est relançable sur une machine avec Python 3.10+ sans packages tiers ni réseau ; le service doit rester local à défaut d'authentification. Les journaux d'échecs initiaux ont été gardés ; chaque incident résolu est lié à un correctif et à une réexécution. La limite de génération de la route HTTP est un garde-fou d'application, tandis que la méthode de génération du modèle est contrôlée par l'appelant et EOS.

8 Conclusions

Le formalisme décrit l'implémentation effective plutôt qu'une proposition d'architecture non codée. Ses équations sont reliées au code et aux tests dans le JSON de preuves. Une version future peut substituer ou combiner d'autres prédicteurs, mais chaque changement devra conserver ou revalider la correspondance entre observables, signatures, fibres, seuils et opérations.

9 Statut du modèle

— **Code intégré** : PASS pour les modules répertoriés.

— **Tests exécutés** : 38/38 tests unitaires et HTTP, syntaxe Python/JavaScript, test DOM navigateur et smoke serveur/CLI réussis dans cet environnement.

— **Propriétés finies vérifiées** : raffinement, cohérence seuil-filtration, minimum double et factorisation sur le domaine sauvegardé.

— **Non démontré** : qualité sémantique générale, raisonnement, factualité, sécurité exhaustive, généralisation hors corpus, absence absolue de bug sur chaque OS ou navigateur.

— **Nature** : prototype statistique autorégressif à n-grammes interpolés, associé à une couche d'empreintes explicites ; pas un Transformer neuronal de grande échelle.